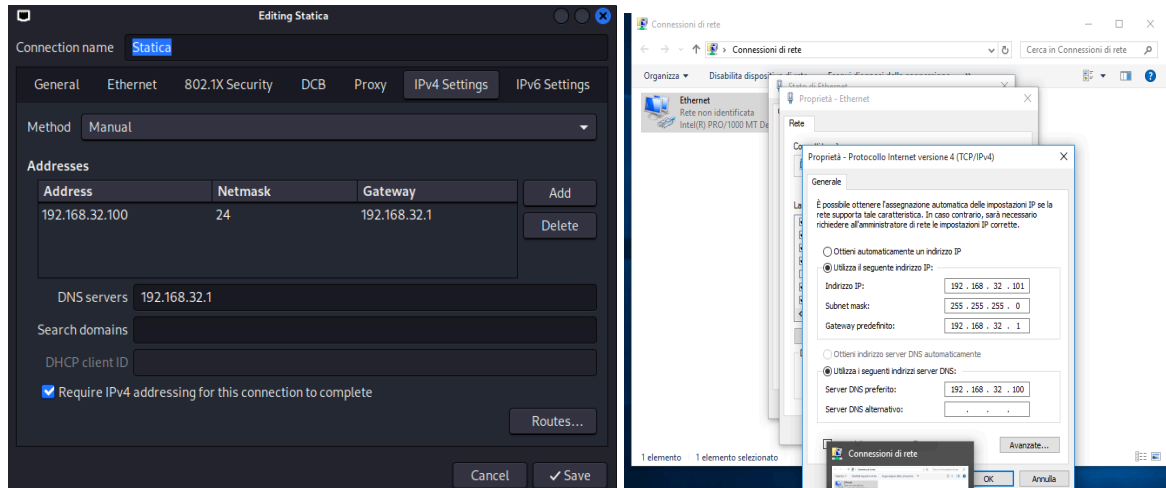


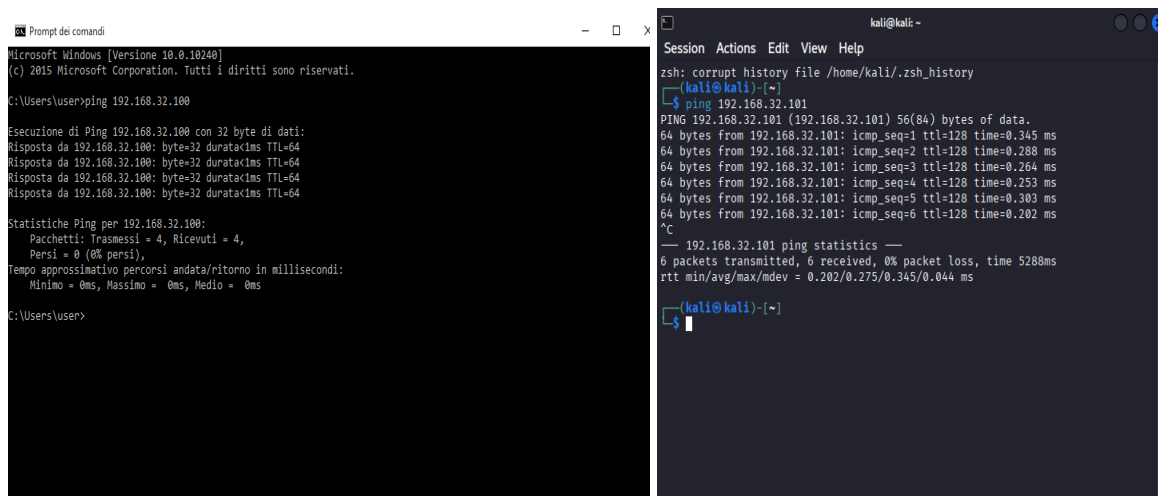
PROGETTO FINALE

Per questo esercizio bisognava utilizzare le due macchine virtuali Kali Linux che mi faceva da server e Windows che mi faceva da client

Per prima cosa ho configurato le due macchine su rete interna, dando a Kali un indirizzo statico 192.168.32.100 e a Windows invece ho dato 192.168.32.101 con DNS 192.168.32.100



Dopo aver configurato gli indirizzi IP, sia da Windows che da Kali ho aperto il terminale e ho fatto un test di connessione per verificare se le due macchine fossero configurate correttamente



Successivamente ho installato il modulo DNS su Kali come spiegato a lezione. Per poter scaricare i pacchetti necessari ho dovuto temporaneamente cambiare la modalità di rete in dinamica (NAT), perché in rete interna non avevo accesso a Internet. Seguendo la procedura vista in classe ho utilizzato CPAN per scaricare e installare il modulo Net-DNS. Durante l'installazione ho riscontrato alcuni errori di permessi, che ho risolto rieseguendo l'operazione con i privilegi di amministratore. Una volta completata correttamente l'installazione, ho riportato la macchina Kali

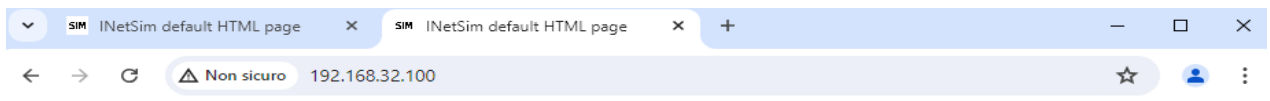
nuovamente in rete interna come richiesto dall'esercizio.

```
kali@kali: ~  
Session Actions Edit View Help  
Installing /usr/local/man/man3/Net::DNS::RR::TSIG.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::TLSA.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::RT.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::NULL.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::NSEC3.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::NAPTR.3pm  
Installing /usr/local/man/man3/Net::DNS::Resolver.3pm  
Installing /usr/local/man/man3/Net::DNS::Question.3pm  
Installing /usr/local/man/man3/Net::DNS::Nameserver.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::CDNSKEY.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::CAA.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::NS.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::MB.3pm  
Installing /usr/local/man/man3/Net::DNS::DomainName.3pm  
Installing /usr/local/man/man3/Net::DNS::Resolver::os2.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::OPENPGPKEY.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::KEY.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::SRV.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::X25.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::TXT.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::LOC.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::INFO.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::A.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::PX.3pm  
Installing /usr/local/man/man3/Net::DNS::RR::EUI48.3pm  
Appending installation info to /usr/local/lib/x86_64-linux-gnu/perl/5.40.1/perlocal.pod  
NLNETLABS/Net-DNS-1.37.tar.gz
```

In seguito ho utilizzato INetSim, come hanno fatto anche altri compagni, per simulare un server web. All'avvio iniziale INetSim ascoltava solo sull'indirizzo locale 127.0.0.1, quindi non era raggiungibile da Windows. Per risolvere il problema ho modificato il file di configurazione impostando come indirizzo di ascolto l'IP di Kali, cioè 192.168.32.100, e poi ho riavviato il servizio. In questo modo sono riuscito a far funzionare INetSim ed ascoltare correttamente sulla rete interna.

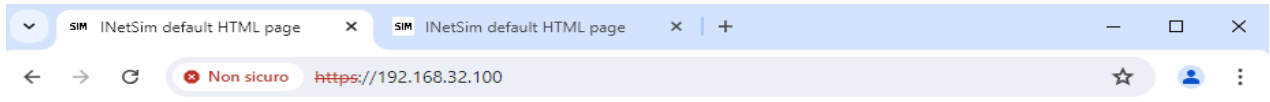
```
(kali@kali)~  
$ sudo inetsim  
NetSim 1.3.2 (2020-05-19) by Matthias Eckert & Thomas Hungenberg  
Using log directory: /var/log/inetsim/  
Using data directory: /var/lib/inetsim/  
Using report directory: /var/log/inetsim/report/  
Using configuration file: /etc/inetsim/inetsim.conf  
Parsing configuration file.  
Configuration file parsed successfully.  
== INetSim main process started (PID 13268) ==  
Session ID: 13268  
Listening on: 192.168.32.100  
Real Date/Time: 2026-01-19 11:45:33  
Fake Date/Time: 2026-01-19 11:45:33 (Delta: 0 seconds)  
Forking services ...  
* http_80_tcp - started (PID 13270)  
* https_443_tcp - started (PID 13271)  
done.  
Simulation running.  
^C * https_443_tcp - stopped (PID 13271)  
* http_80_tcp - stopped (PID 13270)  
Simulation stopped.  
Report written to '/var/log/inetsim/report/report.13268.txt' (24 lines)  
== INetSim main process stopped (PID 13268) ==  
  
$ sudo inetsim  
[sudo] password for kali:  
INetSim 1.3.2 (2020-05-19) by Matthias Eckert & Thomas Hungenberg  
Using log directory: /var/log/inetsim/  
Using data directory: /var/lib/inetsim/  
Using report directory: /var/log/inetsim/report/  
Using configuration file: /etc/inetsim/inetsim.conf  
Parsing configuration file.  
Configuration file parsed successfully.  
== INetSim main process started (PID 2086) ==  
Session ID: 2086  
Listening on: 127.0.0.1  
Real Date/Time: 2026-01-19 11:22:01  
Fake Date/Time: 2026-01-19 11:22:01 (Delta: 0 seconds)  
Forking services ...  
* https_443_tcp - started (PID 2088)  
done.  
Simulation running.  
^C * https_443_tcp - stopped (PID 2088)  
* https_443_tcp - stopped (PID 2088)  
Simulation stopped.  
== INetSim main process stopped (PID 2086) ==
```

A questo punto da Windows ho aperto il browser e ho provato a collegarmi al sito simulato digitando il dominio configurato. La pagina si è caricata correttamente sia utilizzando il protocollo HTTPS che utilizzando HTTP, dimostrando che il servizio web era attivo e funzionante.



This is the default HTML page for INetSim HTTP server fake mode.

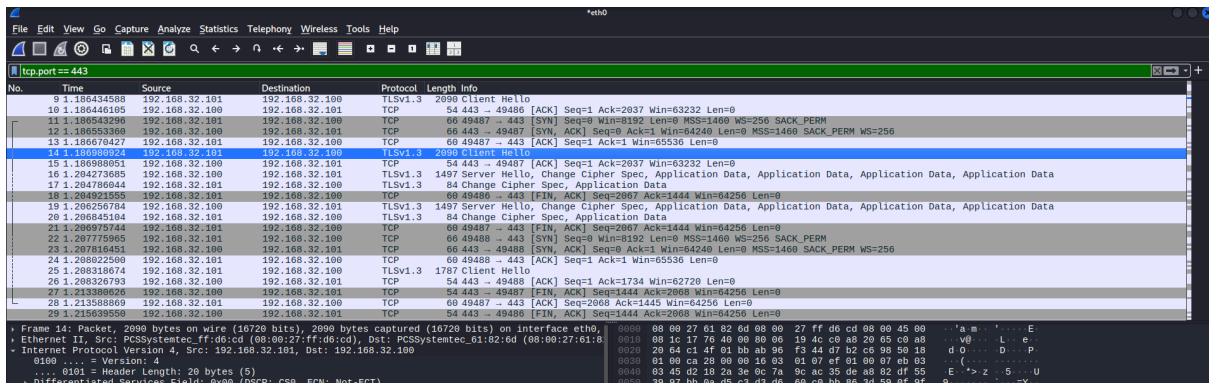
This file is an HTML document.



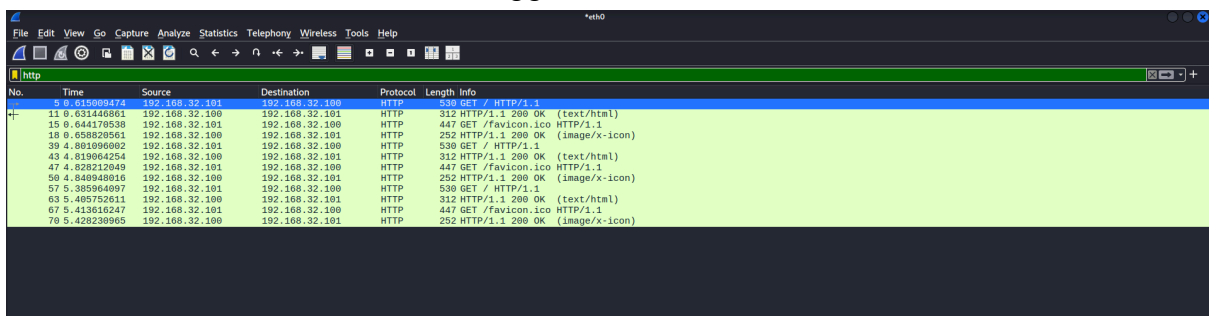
This is the default HTML page for INetSim HTTP server fake mode.

This file is an HTML document.

Infine ho avviato Wireshark sulla macchina Kali per analizzare il traffico di rete. Ho catturato i pacchetti sull'interfaccia di rete e ho applicato un filtro (tcp.port == 443) per osservare il traffico HTTPS sulla porta 443. In questo caso ho potuto vedere la fase di handshake TCP e TLS, con i messaggi di Client Hello e Server Hello, ma non era possibile leggere il contenuto delle comunicazioni perché i dati risultavano cifrati.



Successivamente ho analizzato il traffico HTTP applicando il filtro specifico (HTTP). In questo caso era possibile leggere chiaramente tutte le richieste e le risposte, come le richieste GET, i codici di risposta 200 OK e le informazioni del browser, dimostrando che il traffico HTTP viaggia in chiaro senza cifratura.



STUDENTE: MAZZI EMANUELE